

مستندات فنی و مهندسی: سامانه بی درنگ مدیریت

حریق و نشت گاز

درس: سیستم‌های نهفته و بی درنگ (Embedded and Real-Time Systems)

استاد درس: دکتر سیاوش زرگری

۱. مقدمه و چشم‌انداز معماری

در محیط‌های صنعتی و خانگی مدرن، زمان واکنش به حوادثی نظیر آتش‌سوزی یا نشت گاز، مرز بین کنترل حادثه و بروز یک فاجعه را تعیین می‌کند. تکیه بر نیروی انسانی برای مانیتورینگ مداوم، درصد خطای بالایی دارد و در شرایط بحرانی، سرعت عمل انسان به شدت کاهش می‌یابد. هدف از این پروژه، طراحی یک سیستم تعبیه‌شده (Embedded System) کاملاً خودکار، بلادرنگ (Real-Time) و امنیتی مبتنی بر پردازنده ۳۲ بیتی ARM Cortex-M3 (میکروکنترلر STM32F103C8T6) است.

این سیستم وظیفه دارد به صورت مداوم متغیرهای محیطی را پایش کرده و در صورت بروز خطر، بدون نیاز به دخالت انسان، واکنش‌های فیزیکی سریع (نظیر قطع شیر اصلی گاز یا فعال‌سازی پمپ آب) را انجام دهد. در کنار منطق کنترل خودکار، یک لایه امنیتی یکپارچه (رابط کاربری HMI با کیپد و LCD) برای جلوگیری از دستکاری غیرمجاز سیستم توسط افراد متفرقه طراحی شده است که با یک رفتار زمانی قطعی (Deterministic Timing Behavior) عمل می‌کند.

۲. بیان مسأله و چالش‌های معماری‌های سنتی

سیستم‌های هشداردهنده سنتی و پروژه‌های ساده دانشجویی عموماً از ساختار سرکشی مداوم (Polling) و تاخیرهای مسدودکننده (نظیر دستور HAL_Delay) استفاده می‌کنند. این معماری یک نقطه ضعف اساسی دارد: «خفگی پردازنده».

زمانی که میکروکنترلر در حال انتظار برای فشرده شدن یک کلید یا چاپ یک متن روی نمایشگر است، کاملاً نسبت به سنسورهای حیاتی کور می‌شود. علاوه بر این، در سیستم‌های ساده، اگر دو بحران (مثل حریق و نشت گاز) همزمان رخ دهند، تداخل منطقی ایجاد می‌شود. مسأله اصلی و چالش مهندسی در این پروژه، طراحی معماری پیشرفته‌ای بود که بتواند بدون از دست دادن حتی یک میلی ثانیه، به طور همزمان کیبورد را بخواند، سنسورها را چک کند، وضعیت موتورها را تغییر دهد و رمز عبور را پردازش کند.

۳. کالبدشکافی سخت‌افزار، تخصیص منابع و کالیبراسیون

تخصیص پایه‌ها (Pinout) به صورت ماژولار طراحی شده تا تداخلی بین بخش‌های آنالوگ و دیجیتال پیش نیاید.

۳.۱. شبیه‌سازی سنسورها و رفع خطاهای محاسباتی

الف) سنسور دما (مدل LM35 - متصل به پایه PA0): این سنسور به ازای هر درجه سانتی‌گراد، دقیقاً 10mV خروجی آنالوگ تولید می‌کند. با توجه به اینکه مدل LM35 در پروتئوس با مرجع 5V کار می‌کند، برای فعال‌سازی آلارم در دمای استاندارد 60°C (درجه سانتی‌گراد 600mV)، نیاز به تنظیم آستانه (Threshold) مبدل 12 بیتی (ADC) بود.

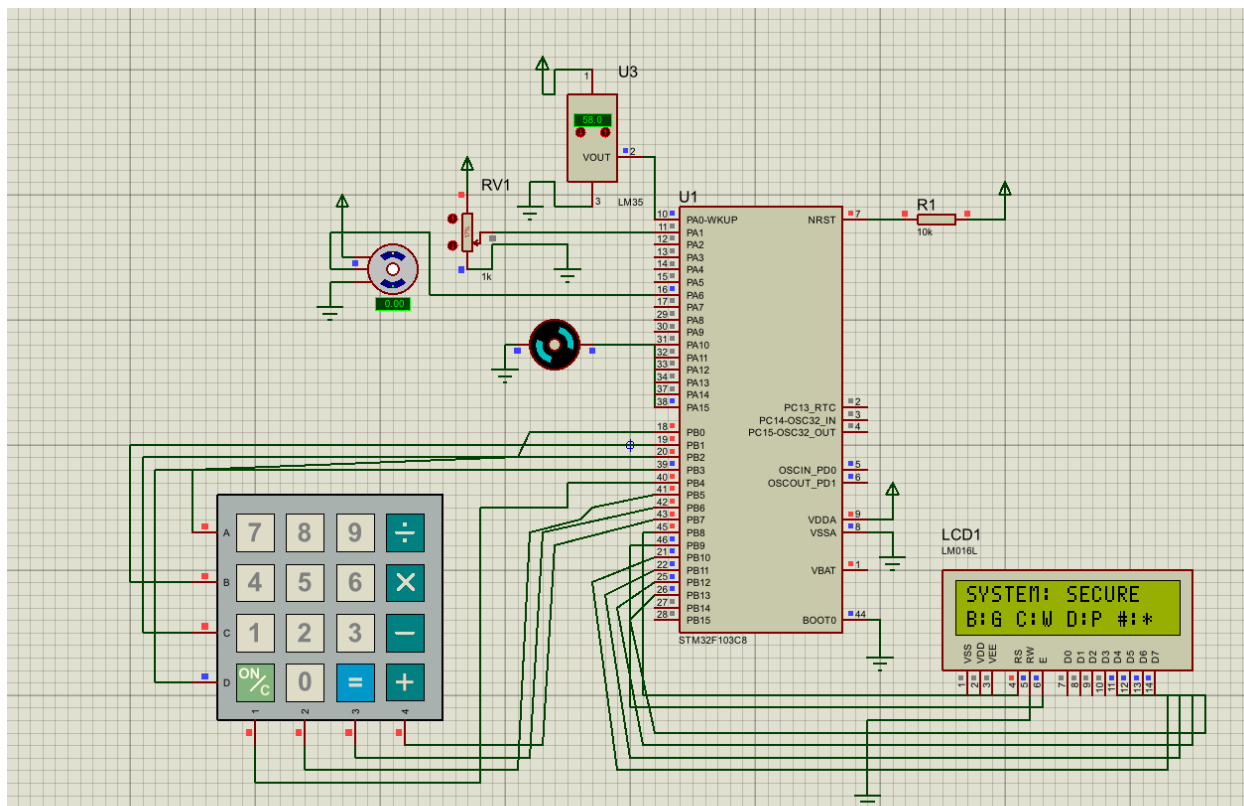
- اصلاح خطای محاسباتی: در ابتدا عدد آستانه بدون محاسبات دقیق روی 1500 تنظیم شده بود که در عمل معادل دمای غیرواقعی بالای 120°C درجه سانتی‌گراد بود و سیستم هرگز وارد فاز حریق نمی‌شد. با اعمال محاسبات دقیق (ارزش هر گام 1.22mV $\approx$)، این عدد کالیبره شد و دقیقاً روی مقیاس 491 تنظیم گردید تا سیستم با پایداری کامل روی دمای 60°C درجه سانتی‌گراد واکنش نشان دهد.

ب) سنسور غلظت گاز (شبیه‌سازی با پتانسیومتر - متصل به پایه PA1): در دنیای واقعی، سنسورهای گاز (مانند سری MQ) بر اساس غلظت گاز، ولتاژ آنالوگ متغیری تولید می‌کنند. در این شبیه‌سازی، از یک پتانسیومتر متصل به پایه PA1 استفاده شد تا افت و خیز ولتاژ یک سنسور گاز واقعی مدل‌سازی شود. آستانه خطر نشت گاز روی 20% مقیاس کامل ADC (عدد دیجیتال خام 819) تنظیم گردید.

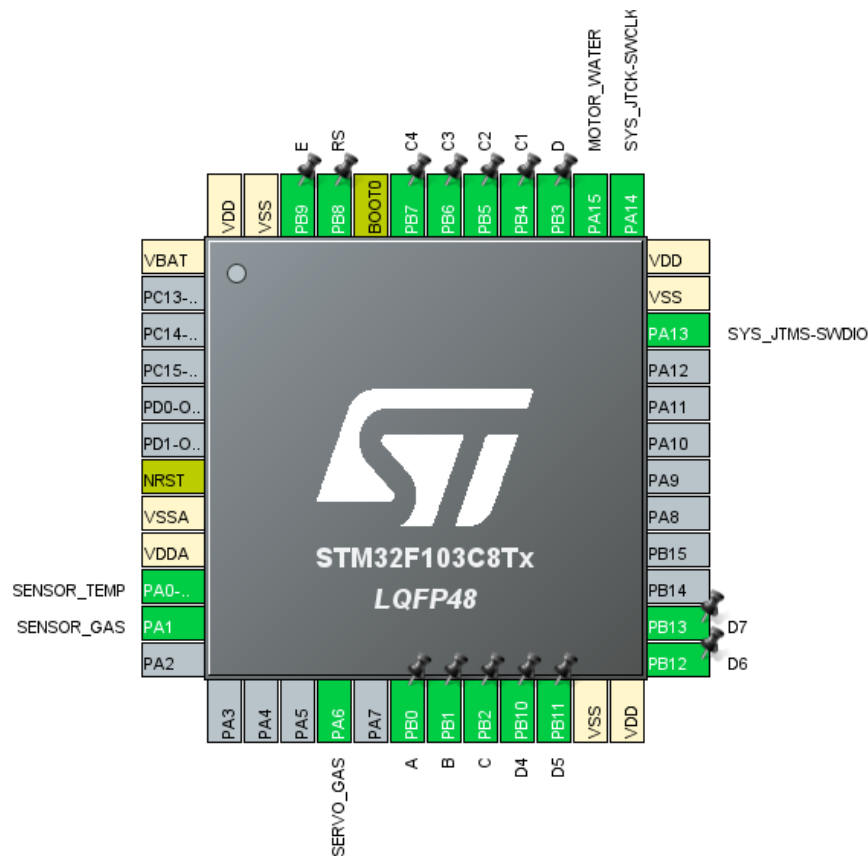
۳.۲. کنترل عملگرهای مکانیکی با سیگنال PWM

برای کنترل موتورهای بدون درگیر کردن پردازنده اصلی، از تایمرهای سخت افزاری استفاده گردید:

- شیر برقی گاز (Servo Motor – پایه PA6): در صنعت، باز و بسته کردن ناگهانی شیرهای گاز خطرناک است و به موتورهای کنترل موقعیت نیاز است. ما تایمر ۳ را با فرکانس پایه ۸ مگاهرتز تغذیه کردیم. با تنظیم Prescaler روی ۷۹ و ARR روی ۱۹۹۹، دقیقاً به فرکانس کاری ۵۰ هرتز (دوره تناوب ۲۰ میلی ثانیه) رسیدیم. با کالیبراسیون دقیق (و تنظیم Minimum Angle در پروتئوس روی 0.0)، شیر در حالت ایمن روی زاویه ۰ درجه (پالس ۱ میلی ثانیه معادل مقدار رجیستر ۱۰۰) و در حالت خطر دقیقاً روی زاویه ۹۰ درجه (پالس ۲ میلی ثانیه معادل رجیستر ۲۰۰) قفل می شود تا شیر فیزیکی کاملاً مسدود گردد.
- پمپ آب (DC Motor – پایه PA15): با استفاده از کانال ۱ تایمر ۲ کنترل می شود که در زمان حریق، با حداکثر توان (پالس ۹۹۹) فعال می گردد.



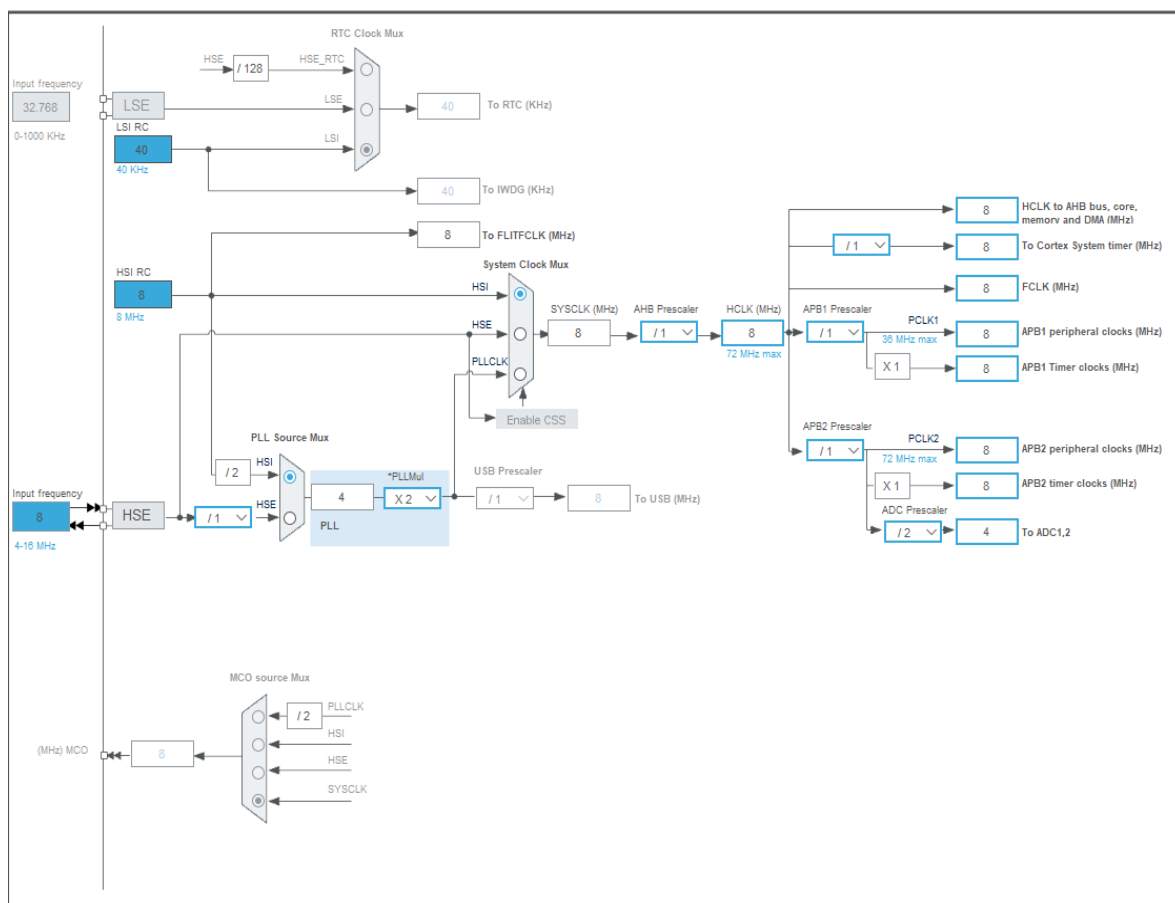
شمارتیک کامل پروتئوس



Pinout میکروکنترلر در نرم افزار CubeMX

۴. تنظیمات کلاک و زمان بندی (Clock Tree)

تولید زمان بندی دقیق در سیستم های تعبیه شده، وابستگی مطلق به انتخاب صحیح منبع کلاک دارد. مغز متفکر سیستم، میکروکنترلر STM32F103C8 است. ما کلاک سیستم را روی نوسان ساز داخلی سرعت بالا (HSI) با فرکانس دقیق ۸ مگاهرتز (8 MHz) تنظیم کردیم. این فرکانس پایه، مرجع تمام محاسبات زمانی، تایمرها و وقفه های ما در کل پروژه است. از آنجا که نیازمندی های سیستم با این فرکانس به طور کامل برآورده می شود، استفاده از واحد PLL برای افزایش فرکانس ضرورتی نداشت و پایداری شبیه سازی را در تولید فرکانس دقیق ۵۰ هرتز تضمین کرد.



تنظیمات Clock Tree در نرم افزار CubeMX

۵. معماری ورودی بی درنگ (Real-Time Input) و اسکن کپید

برای جلوگیری از نویز، ستون‌های کپید ۴x۴ به پایه‌های ورودی PB4 تا PB7 (با مقاومت Pull-Up داخلی) و سطرها به خروجی‌های PB0 تا PB3 متصل شدند.

۵.۱ چرا کپید با وقفه خارجی (EXTI) پیاده‌سازی نشد؟

یک اشتباه رایج، اتصال پایه‌های کپید به وقفه خارجی (External Interrupt) است. کلیدهای مکانیکی دارای پدیده «لرزش کنتاکت» یا Bounce هستند. اگر از EXTI استفاده می‌کردیم، فشردن یک کلید باعث ده‌ها وقفه پشت‌سرهم شده و پردازنده را فلج می‌کرد.

۵.۲ راهکار مهندسی: اسکن غیرمستودکننده با تایمر ۴ (TIM4)

ما تایمر ۴ (TIM4) را طوری تنظیم کردیم که هر ۱۰ میلی ثانیه یک وقفه (Interrupt) ایجاد کند. پردازنده هر ۱۰ میلی ثانیه کار اصلی خود را متوقف کرده، در حد چند میکروثانیه فقط یک سطر از کپی را چک می کند و سریعاً به خواندن سنسورها برمی گردد. این تاخیر ۱۰ میلی ثانیه ای تعمداً، به موتور فیزیکی پروتئوس و کلید مکانیکی فرصت می دهد تا افت ولتاژ روی سیم ها را محاسبه کند. آستانه رفع لرزش کلید (Debounce): برای حذف لرزش کلید، سیستم شمارنده ای هوشمند طراحی شد که باید ۱۵ چرخه متوالی (معادل ۱۵۰ میلی ثانیه) وضعیت «عدم فشردن کلید» را رصد کند تا حافظه کلید قبلی پاک شود. این زمان بندی دقیق باعث می شود هر چقدر هم دکمه را نگه دارید، عدد فقط یک بار تایپ شود و سیستم کاملاً بلادرنگ (Real-Time) بماند.

۶. عملکرد کلیدهای کپد و بازخورد منوها روی LCD (مسیریابی وابسته به حالت)

کپد ماتریسی در این پروژه به عنوان رابط اصلی انسان و ماشین (HMI) عمل می کند. برای بالا بردن ضریب اطمینان و جلوگیری از خطای اپراتور، منطق نرم افزاری این بخش به صورت مسیریابی وابسته به حالت (State-Dependent Routing) طراحی شده است. این یعنی یک کلید مشخص، بر اساس وضعیت فعلی موتورها، منوی متفاوتی را فراخوانی می کند:

- کلید A (لغو عمومی و بازگشت): در هر منویی (به جز وضعیت آلارم های اصلی) اگر کاربر کلید A را فشار دهد، سیستم بلافاصله عملیات را لغو می کند و با نمایش پیام CANCELED... به صفحه اصلی (SYSTEM: SECURE) برمی گردد.
- کلید B (منوی دوگانه شیر گاز): عملکرد این کلید پویاست. اگر شیر گاز باز باشد، منوی SHUT GAS [A=Cncl] (بستن گاز) ظاهر می شود. اگر شیر بسته باشد، منوی OPEN GAS ظاهر می شود. پس از تایید رمز عبور ۴ رقمی (که با * ماسک می شود)، خروجی اعمال شده و پیام GAS VLV: CLOSED / OPEN چاپ می گردد.
- کلید C (منوی دوگانه پمپ آب): مشابه شیر گاز، وضعیت موتور آب به صورت پویا چک می شود. اگر پمپ خاموش باشد منوی STRT WTR و اگر روشن باشد منوی STOP WTR فعال می شود.

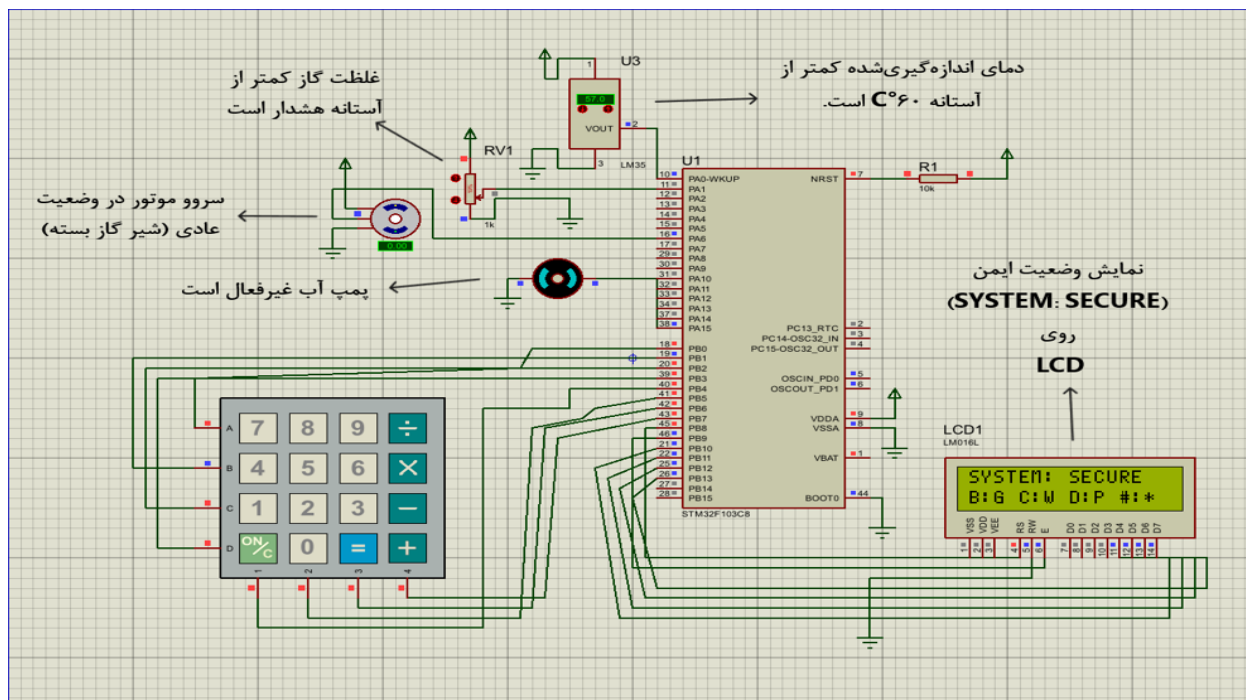
- کلید D (تغییر رمز دو مرحله‌ای): ابتدا رمز فعلی را می‌پرسد و در صورت تایید، امکان درج و ذخیره رمز جدید را فراهم می‌کند.
- کلیدهای نمایش زنده (* و #): با فشردن کلید * دمای زنده محیط (به سانتی‌گراد) و با کلید # غلظت گاز (به درصد) با تبدیل فوری روی LCD نمایش داده می‌شود.

۷. منطق اولویت بحران (Priority Logic)

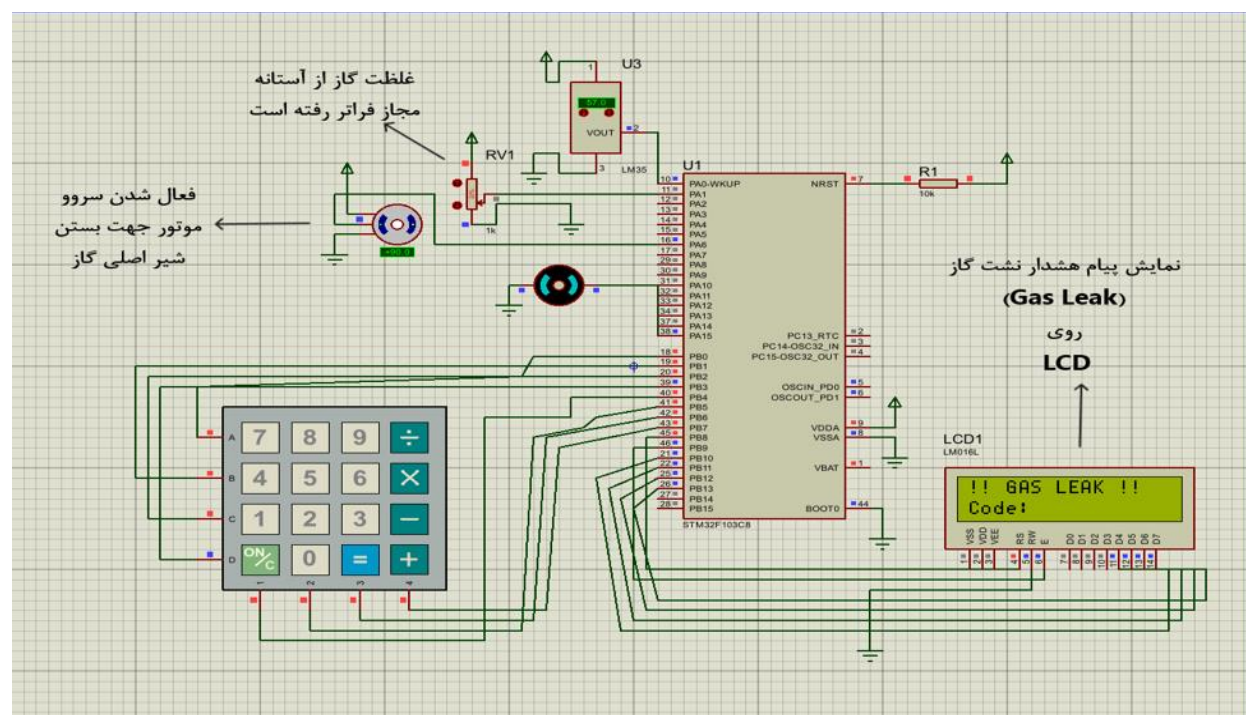
سیستم دارای یک ساختار اولویت‌بندی صنعتی و قطعی است تا در صورت وقوع همزمان چند بحران، تداخل منطقی در فرمان عملگرها ایجاد نشود:

- اولویت دوم (نشت گاز): در صورت نشت گاز (عبور از مرز ۲۰٪)، فقط شیر گاز بسته می‌شود.
- اولویت اول (بحران حریق): در صورت تشخیص دمای بالای ۶۰ درجه، سیستم هم پمپ آب را روشن می‌کند و هم برای جلوگیری از انفجار، شیر گاز را می‌بندد. اگر حین نشت گاز، حریق هم رخ دهد، سیستم بلافاصله پرچم گاز را نادیده گرفته و مستقیماً به اولویت اول ارتقا می‌یابد.

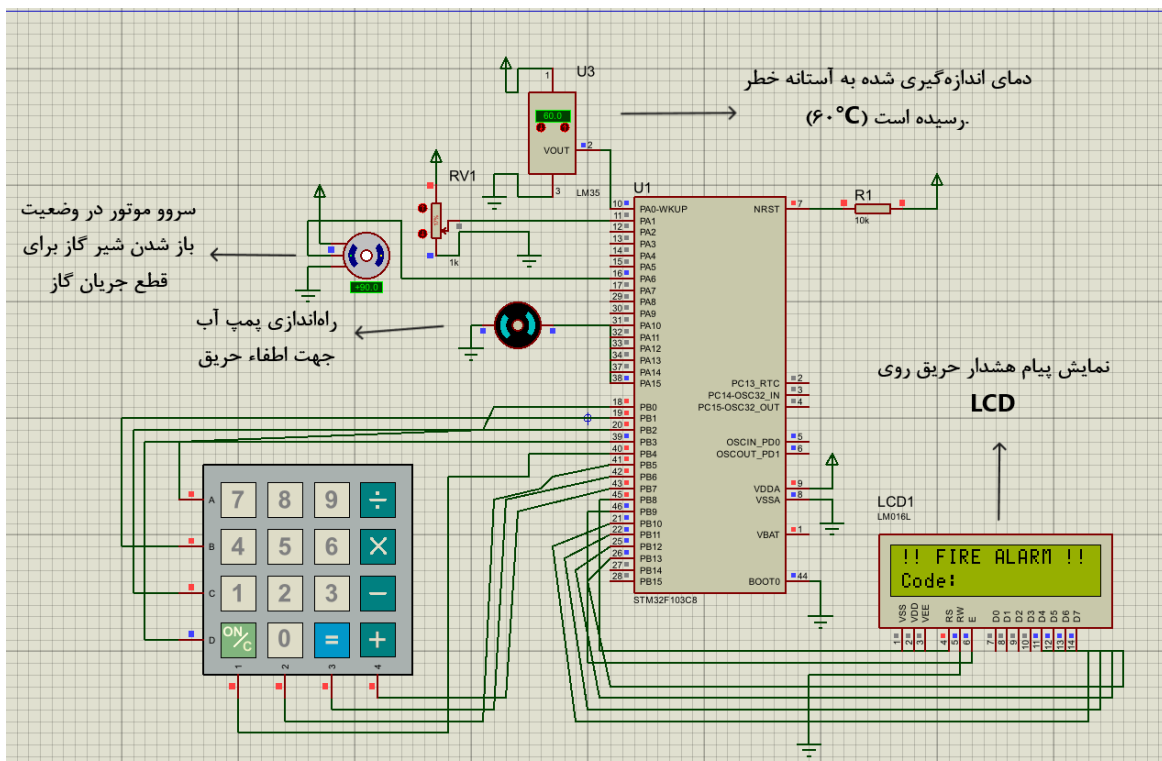
رویداد تشخیص داده شده	سطح اولویت	فرمان پمپ آب	فرمان شیر برقی گاز
نشت گاز مستقل (گذر از ۲۰٪)	اولویت ۲ (متوسط)	خاموش (0٪) (PWM)	بسته قفل شده (زاویه ۹۰°)
بحران حریق (گذر از ۶۰°C)	اولویت ۱ (بحرانی)	روشن (100٪) (PWM)	بسته قفل شده (زاویه ۹۰°)
نشت گاز + حریق همزمان	اولویت ۱ (بحرانی)	روشن (100٪) (PWM)	بسته قفل شده (زاویه ۹۰°)



تصویر از شبیه سازی پروتئوس در حالت عادی (System Secure)



تصویر از شبیه سازی پروتئوس در حالت نشت گاز (GAS ALARM)



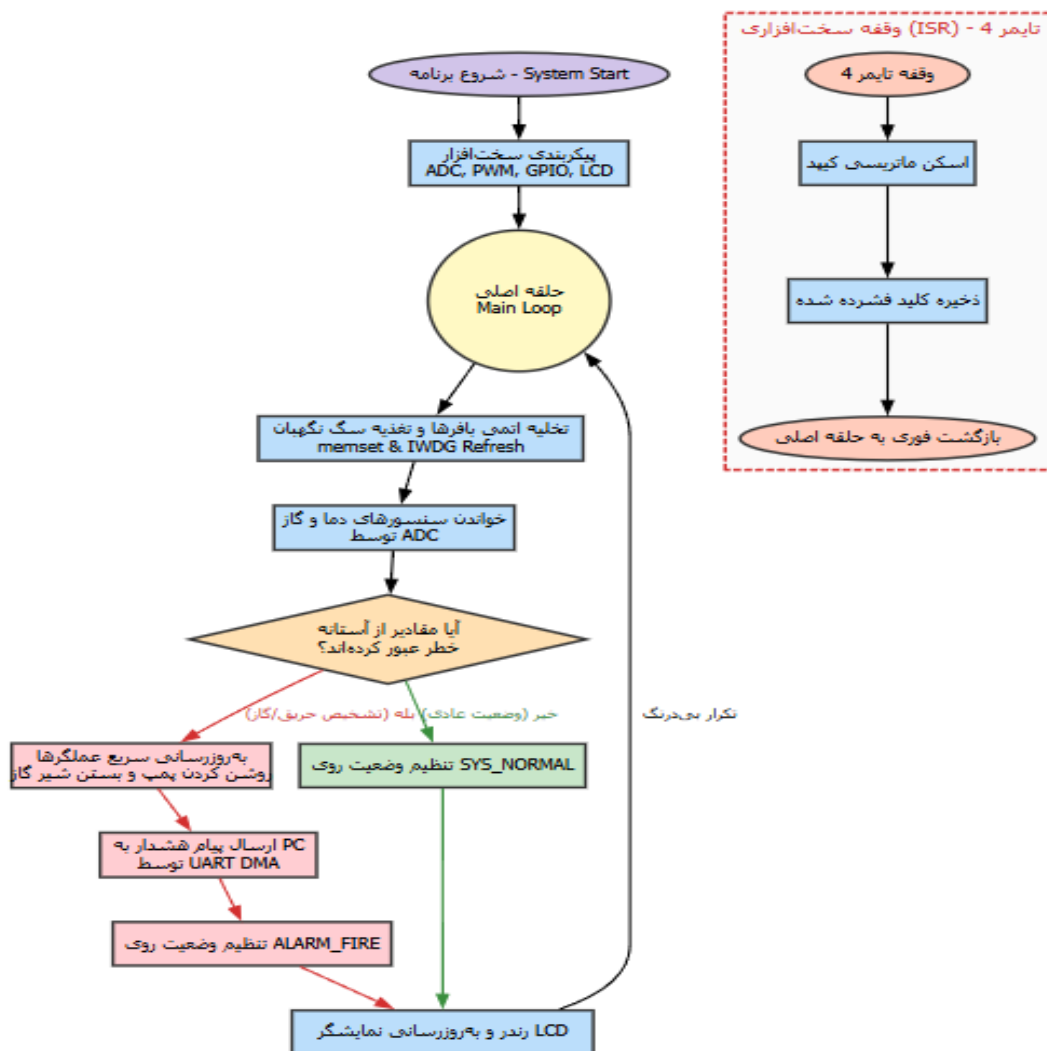
تصویر از شبیه سازی پروتوس در حالت حریق (FIRE ALARM)

۸. معماری بلادرنگ Bare-Metal و سیستم وقفه‌های سخت‌افزاری

یک تصور اشتباه مهندسی این است که برای داشتن یک سیستم بلادرنگ (Real-Time)، حتماً باید یک سیستم عامل مثل FreeRTOS نصب باشد. ما قصد داشتیم سیستم را بر بستر تسک‌های زمان‌بندی شده سیستم عامل سوار کنیم، اما برای جلوگیری از سربار (Overhead) غیر ضروری پردازنده، معماری ماشین حالت Bare-Metal جایگزین شد.

مفهوم وقفه در این پروژه: در کدهای معمولی، میکروکنترلر در یک حلقه دائم می‌چرخد و دکمه‌ها را چک می‌کند؛ این کار پردازنده را فلج می‌کند. در این پروژه از واحد کنترل کننده وقفه‌ها (NVIC) استفاده کردیم. پردازنده ۱۰۰٪ وقت خود را در حلقه اصلی صرف خواندن سنسورهای آنالوگ می‌کند. به محض اینکه تایمر ۴ علامت بدهد، پردازنده در یک میکروثانیه کار خود را رها کرده، کپی را بررسی می‌کند و فوراً به حلقه اصلی برمی‌گردد. به همین دلیل، سنسورها در هر ثانیه هزاران بار بدون کوچک‌ترین لگ نرم‌افزاری خوانده می‌شوند و واکنش به حریق کاملاً آنی است.

فلوچارت حلقه اصلی (Main Loop Flowchart):



تخلیه اتمی حافظه (Atomic Data Flushing): برای حل مشکل تداخل رم‌ها و نمایش پیام خطا در زمان جابجایی بین منوها، دستور `memset(input_buffer, 0, NUMBER_OF_KEYS)` در ابتدای ورود به هر منو قرار داده شد. این دستور به صورت یکپارچه کل آرایه پسورد را پیش از تایپ کاربر پاک‌سازی می‌کند تا کاراکترهای باقیمانده، محاسبات را خراب نکنند.

۹. چالش‌های فنی، دیباگینگ پیشرفته و نحوه حل آن‌ها

در طول توسعه پروژه، با چالش‌های فنی متعددی در سطح رجیسترها و فیزیک شبیه‌ساز مواجه شدیم که با اصول مهندسی برطرف گردیدند:

الف) خفگی پردازنده در پورت سریال (UART) و تصمیم به حذف آن: در فازهای اولیه، از ترمینال کامپیوتر (UART) استفاده می‌کردیم. قرار دادن دستورات سنگین چاپ متن (HAL_UART_Transmit) درون تابع وقفه گیرنده (Rx-Callback) باعث می‌شد پردازنده تمام پهنای باند خود را صرف ارسال کاراکترها کند و کاراکترهای بعدی رمز عبور کیبورد از دست بروند (Data Loss) و سیستم قفل کند. ابتدا با تبدیل وقفه به حالت سبک‌وزن (Lightweight) و انتقال پردازش به حلقه اصلی مشکل را کمرنگ کردیم، اما برای تضمین قطعی بودن زمان پاسخگویی (WCRT)، تصمیم گرفتیم پورت سریال را کاملاً از معماری نهایی حذف کرده و به کیپد و LCD متکی شویم.

ب) پدیده هذیان الکتریکی و فرار فیزیکی کیپد: پس از حذف پورت سریال و اضافه کردن کیپد و LCD موازی به پورت B، سیستم دچار "هذیان الکتریکی" شد. بدون لمس کلید، کاراکترها روی LCD نقش می‌بستند و ستون‌های کیپد در پروتئوس به رنگ خاکستری (حالت شناور یا Floating) در می‌آمدند.

- روش حل: ریشه اول مشکل، پایه‌های معلق بود که با فعال‌سازی نرم‌افزاری مقاومت‌های بالاکش داخلی (GPIO_PULLUP) و لتاژ آن‌ها روی ۵ ولت فیکس شد. ریشه دوم سرعت بالا بود که الگوریتم را به "اسکن تک‌سطر در هر ۱۰ میلی‌ثانیه" تغییر دادیم تا موتور پروتئوس فرصت پردازش افت و لتاژ را داشته باشد.

ج) عدم پاسخگویی و قفل شدن رجیسترهای سایه (Shadow Registers): سروو موتور در محیط شبیه‌ساز هیچ واکنشی به مقادیر جدید رجیستر CCR نشان نمی‌داد و روی صفر قفل بود.

- روش حل: با بررسی دیتاشیت مشخص شد مقادیر سایه در تایمرهای STM32 تا زمان رخ دادن رخداد به‌روزرسانی (Update Event) اعمال نمی‌شوند. با اضافه کردن دستور سخت‌افزاری `htim3.Instance->EGR = TIM_EGR_UG`; کلاک تایمر وادار به بارگذاری فوری مقادیر شد و موتور به کار افتاد.

د) قفل شدن زاویه سروو موتور در 0.18 درجه: به دلیل خطای گرد کردن (Rounding) در پروتئوس، پالس PWM به صورت زاویه کاملاً صفر دیده نمی‌شد.

- روش حل: تنظیمات Minimum Angle قطعه در پروتئوس روی 0.0 تنظیم شد و در کد، مقدار مقایسه روی 100 قرار گرفت تا سیستم کاملاً کالیبره شود.

۱۰. تحلیل زمان‌بندی (Timing Analysis) و برآورد WCET/WCRT

مقادیر جدول زیر، بر اساس تحلیل معماری و تخمین تئوری در فرکانس 8MHz ارائه شده‌اند و حاصل اندازه‌گیری مستقیم تجربی نمی‌باشند.

ماژول / عملیات (Task)	رویکرد اجرایی	تخمین زمان اجرا (WCET)	تأثیر بر زمان پاسخ‌گویی سیستم
تبدیل ADC	Polling	~15 us	بسیار پایین؛ تأخیری ایجاد نمی‌کند.
اسکن کیپد	ISR (TIM4)	~3 us	اجرای روتین در حد میکروثانیه است.
تولید PWM	سخت‌افزاری	N/A	بار پردازشی ناچیز (به‌روزرسانی CCR).
ارزیابی FSM	نرم‌افزاری	~20 us	پردازش شروط منطقی بسیار سریع است.
به‌روزرسانی LCD	Bounded Blocking	~40 us / Char	مسدودسازی بسیار کوتاه؛ صرفاً هنگام تغییر منوها.

برآورد تئوری زمان پاسخ‌گویی (WCRT):

انتظار می‌رود زمان پاسخ‌گویی سیستم (WCRT) در مرتبه چند صد میکروثانیه تا کمتر از یک میلی‌ثانیه قرار گیرد. در زمان تغییر وضعیت بحرانی، ابتدا ثبات مقایسه‌ای عملگرها (CCR) به‌روزرسانی شده و سپس تابع رندر LCD فراخوانی می‌شود تا تأخیر نمایشگر، بر WCRT عملگرهای حیاتی تأثیری نداشته باشد.

۱۱. تحلیل حالت‌های خرابی (FMEA) و منابع سیستم

بر اساس الزامات ایمنی (Fail-Safe)، واکنش سیستم به خرابی‌ها تحلیل گردید:

- قطع ارتباط سنسورها: مقاومت‌های پایین‌کش سخت‌افزاری، ولتاژ را به صفر می‌رسانند تا از آلارم کاذب جلوگیری شود. در نسخه صنعتی، پیشنهاد می‌شود منطق تشخیص نقص سنسور (Sensor Fault) برای مقادیر صفر مطلق نیز به ماشین حالت افزوده شود.
- قفل‌شدگی مکانیکی عملگرها: در صورت قفل شدن شیر گاز، سیستم به دلیل نداشتن فیدبک موقعیت (Encoder) سیگنال PWM را پیوسته ارسال می‌کند تا به محض رفع گیر، شیر مسدود شود.
- تحلیل مصرف رم (SRAM): بر اساس برآورد استاتیک متغیرهای سراسری و عدم استفاده از RTOS، مصرف حافظه SRAM کمتر از ۱ کیلوبایت تخمین زده می‌شود.

۱۲. ماتریس آزمون و اعتبارسنجی

عملکرد منطق نرم‌افزاری تحت سناریوهای تست زیر ارزیابی گردید:

شناسه	شرح سناریو (Test Case)	رفتار مورد انتظار (Expected)	نتیجه واقعی (Actual)	وضعیت
TC-01	ولتاژ سنسور LM35 فراتر از mV600	صدور پالس ۱۰۰٪ برای پمپ و زاویه ۹۰° برای سروو	پمپ فعال شد، سروو در ۹۰° قفل شد.	Pass
TC-02	ولتاژ گاز فراتر از آستانه ۸۱۹	تنظیم سروو روی ۹۰° و توقف پمپ آب	سروو بسته شد، پمپ خاموش ماند.	Pass
TC-03	وقوع نشت گاز، و بلافاصله حریق	ارتقای وضعیت سیستم به حریق	پرچم گاز Override شد، پمپ روشن گردید.	Pass
TC-04	فشاردن سریع کلیدهای کیپد	ثبات تنها یک کاراکتر	شمارنده ms۱۵۰ مانع از ثبت نویز شد.	Pass

۱۳. مواردی که مدنظر بود اما اجرا نشد (محدودیت‌ها و توسعه آینده)

در طول پیشبرد پروژه ایده‌های پیشرفته‌ای مدنظر گروه بود که به دلایل فنی و محدودیت‌های نرم‌افزاری شبیه‌ساز، پیاده‌سازی آن‌ها به فازهای توسعه آینده موکول شد:

۱. عدم موفقیت در راه‌اندازی نمایشگر به صورت I2C: هدف اولیه ما کاهش سیم‌کشی‌های LCD از ۶ پین به ۲ پین با استفاده از ماژول درایور PCF8574 تحت پروتکل I2C بود. اما توابع درایور سخت‌افزاری (HAL_I2C_Master_Transmit) به دلیل عدم پشتیبانی کامل شبیه‌ساز پروتئوس از خاصیت کشیدگی کلاک (Clock Stretching) و تایمینگ‌های رجیستری STM32، در یک حلقه بی‌پایان وضعیت HAL_BUSY گیر

می کردند و میکروکنترلر فریز می شد. در نتیجه برای حفظ پایداری، از پروتکل موازی استاندارد استفاده شد.

II. عدم استفاده از قابلیت سگ نگهبان آنالوگ (Analog Watchdog): در نظر داشتیم به جای بررسی مداوم مقادیر سنسورها در حلقه اصلی while (1)، از قابلیت سخت افزاری سگ نگهبان آنالوگ در واحد ADC استفاده کنیم تا خود سخت افزار در صورت عبور ولتاژ از حد مجاز، مستقیماً وقفه تولید کند. با توجه به ناپایداری شبیه ساز در مدیریت کلاک این واحد، پیاده سازی نرم افزاری در حلقه اصلی به عنوان یک راه حل جایگزین و پایدارتر انتخاب گردید.

III. عدم پیاده سازی سیستم عامل بلادرنگ (FreeRTOS): قصد داشتیم سیستم را بر بستر تسک های زمان بندی شده سیستم عامل سوار کنیم. اما با توجه به ماهیت پروژه و برای جلوگیری از سربار (Overhead) غیر ضروری پردازنده، معماری ماشین حالت Bare-Metal طراحی شد که با سرعت میکروثانه به خوبی پاسخگوی نیاز سیستم بود.

۱۴. نتیجه گیری نهایی و چشم انداز

این پروژه اثبات کرد که تفاوت بسیار زیادی بین کدنویسی ساده و «مهندسی سیستم های تعبیه شده» وجود دارد. ما موفق شدیم با بهره گیری از توانمندی های سخت افزاری پردازنده های ARM Cortex-M3 (نظیر تایمرهای پیشرفته، وقفه ها و مقاومت های داخلی)، سیستمی را خلق کنیم که به جای پردازش خطی و کند، به صورت موازی (Parallel) می اندیشد.

چالش هایی که در مسیر همگام سازی کدهای C با فیزیک شبیه ساز پروتئوس داشتیم، دیدگاه ما را نسبت به زمان بندی (Timing) در ابعاد میلی ثانیه و میکروثانه، مدیریت حافظه RAM و مدیریت جریان برنامه بسیار عمیق تر کرد. سیستم نهایی نه تنها یک هشداردهنده حریق، بلکه یک پنل صنعتی HMI کامل با قابلیت های امنیتی، کنترل دستی و واکنش بلادرنگ به خطرات است.

۱۵. واژه نامه (Glossary)

- WCRT (Worst-Case Response Time): طولانی ترین زمان ممکن از لحظه وقوع رویداد خارجی تا آغاز واکنش سیستم.

- **WCET (Worst-Case Execution Time)**: بیشترین زمان مورد نیاز پردازنده برای اجرای کامل یک بلوک از کد.
- **ISR (Interrupt Service Routine)**: روتین خدمات وقفه.
- **FSM (Finite State Machine)**: ماشین حالت متناهی.
- **FMEA (Failure Mode and Effects Analysis)**: تحلیل حالت‌های خرابی سیستم.
- **Bare-Metal**: برنامه‌نویسی مستقیم منابع سخت‌افزاری میکروکنترلر، بدون سیستم‌عامل.

۱۶. مشارکت اعضا در پروژه

- **نوید رستمی**: طراحی ماشین‌های حالت سیستم و یکپارچه‌سازی کل پروژه، طراحی و راه‌اندازی نمایشگر و کیپد (توابع منطقی و امنیتی + اسکن)، طراحی پروتئوس پروژه
- **معید غیادی**: طراحی سنسور دما و پمپ آب، طراحی سنسور گاز و شیر گاز، طراحی پاورپوینت پروژه و طراحی گزارش کار